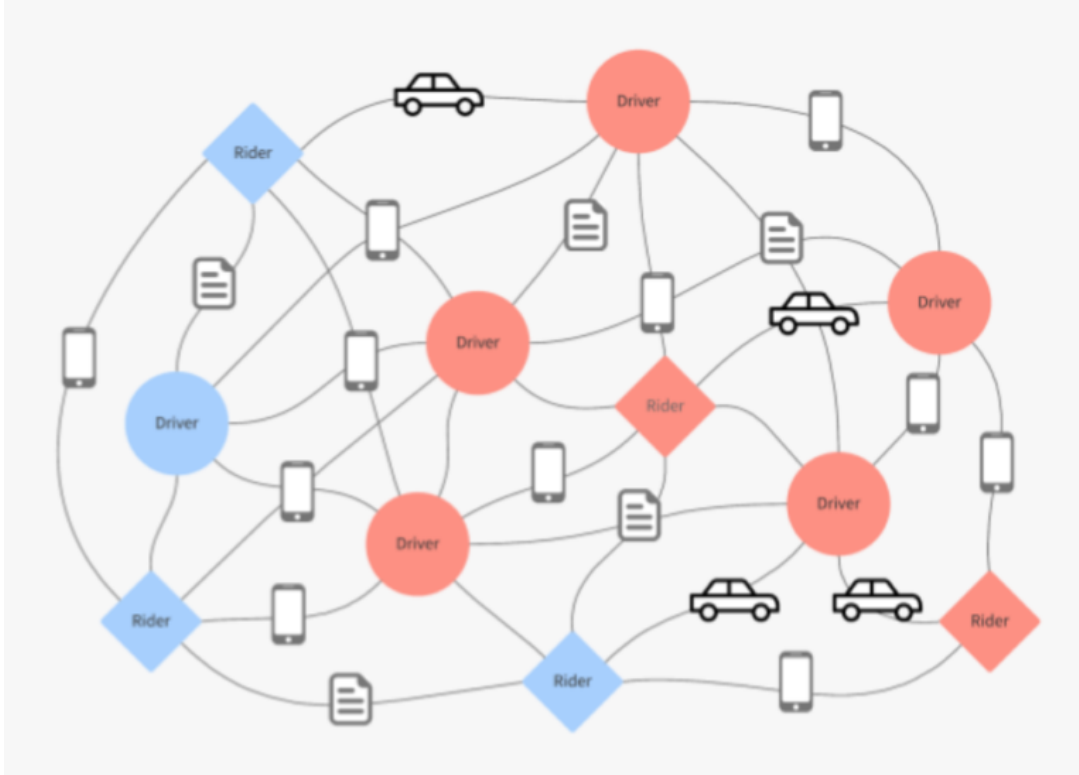


Fraud Detection: Using Relational Graph Learning to Detect Collusion

Xinyu Hu, Chengliang Yang, Lawrence Murray, Ankur Sarda, Ankit Jain, and Piero Molino

May 21, 2021



As Uber grew in popularity and scale among legitimate customers, it also attracted the attention of financial criminals in the cyberspace. One type of fraudulent behavior is collusion, a cooperative fraud action among users. For example, users collude by taking fake trips with stolen credit cards resulting in chargeback (a bank-initiated refund for a credit card purchase). In this article we show a case study of applying a cutting-edge, deep graph learning model called *relational graph convolutional networks* (RGCN) [1] to detect such collusion.

Graph learning methods have been extensively used in fraud detection [2] and recommendation tasks [3]. For example, at Uber Eats, a graph learning technique has been developed to surface the foods that are most likely to appeal to an individual user [4]. Graph learning is one of the ways to improve the quality and relevance of our food and restaurant recommendations on the Uber platform. A similar technology can be applied to detect collusion. Fraudulent users are often connected and clustered, as shown in Figure 1, which can help detection. We outline a case study of a relational graph learning model that uses this information to detect colluding users, and uses different connection types to improve learning. The purpose is to share our findings in this case study which can be generalized to solving other related fraud detection problems. The model developed in this article is not used in Uber's production platform.



Engineering

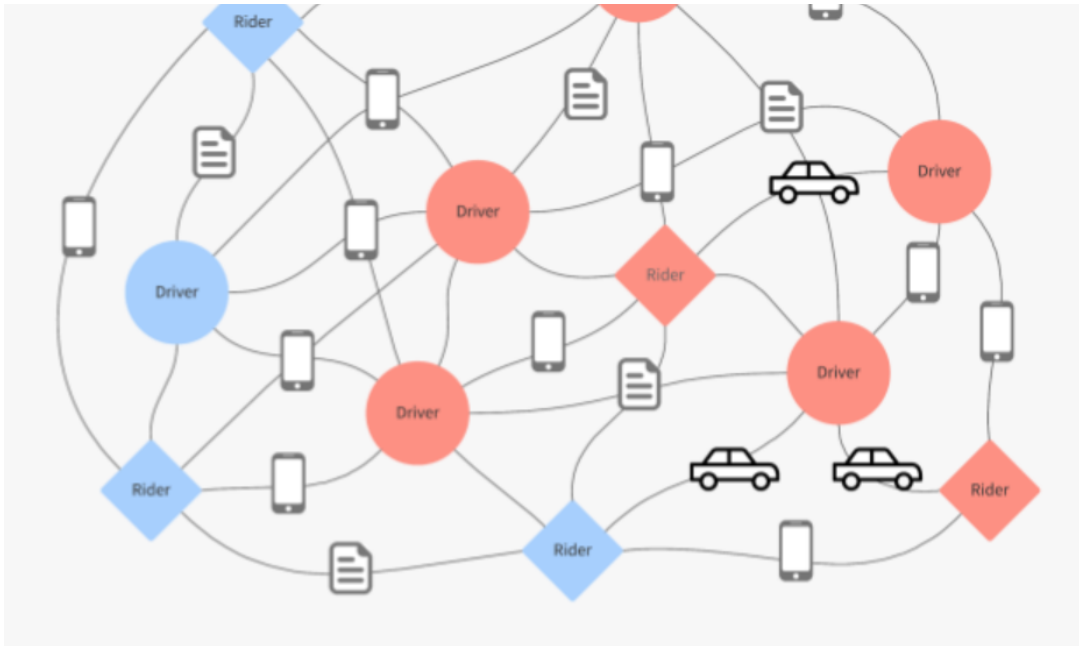


Figure 1: An illustrated graph of connected drivers and riders. Red nodes represent fraud users, blue nodes represent legitimate users. Users are connected via shared information.

Relational Graph Learning

We applied an RGCN model on a small sample of data to predict if users are committing fraud. In the user graph, there are two types of nodes: drivers and riders. Drivers and riders can be connected via shared information. Each user can be viewed as a node in a graph, which is represented by a vector of embeddings. This representation encodes the properties of users and their neighboring community, and can be easily used for machine learning tasks, such as node classification and edge prediction. For example, to detect whether a user is fraudulent or not, we use not only the user's features, but also features from neighboring users within several hops. The model is based on neural networks operating on graphs, developed specifically to model multi-relational graph data. This type of graph learning has been shown to lead to significant improvements in node classification [5].

Users on one platform are connected to each other via shared information. We found that distinguishing between different connection types amplifies the signal for fraud detection. Therefore we use connection type as a feature for graph learning.

To better understand how we model graph user data and detect collusion, it helps to know the basics of RGCNs. Graph convolutional networks (GCN) have been shown to be very effective at encoding features from structured neighborhoods [6], in which the same weight is assigned to edges connected to the source node. RGCN, on the other hand, has relation-specific transformations that depend on the type and direction of an edge. Therefore, the message calculated for each node is augmented with edge type information. Figure 2 shows a diagram of a RGCN model. The inputs of the model consist of node features and edge types. Node features are passed to a RGCN layer then transformed into a vector of learned representations by aggregating the learned representations from connected neighbors. The messages coming



Engineering

activation function (such as the ReLU). RGCN layers can extract high-level node representations by message passing and graph convolution. With a softmax layer as the output layer and cross-entropy as the loss function, RGCN models are able to learn node scores.

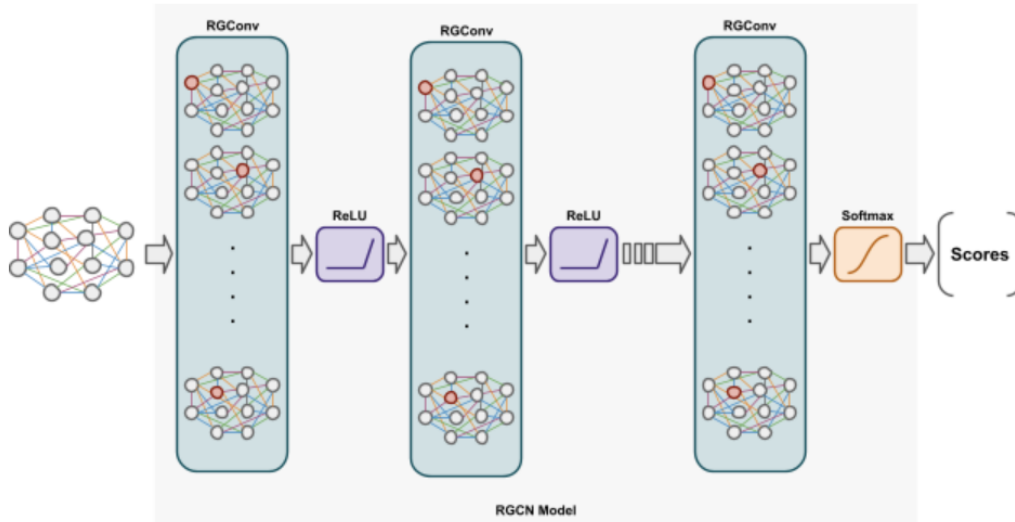


Figure 2 Modeling flow of RGCN: inputs of users' node features and edge types are passed into several RGCN layers to generate node scores. The edge colors represent different edge types.

The transformed feature vectors of neighboring nodes depend on edge-specific transformations that note the type and direction of an edge. A single self-connection is added as a special edge type to each node, so that the representation of a node at layer $l+1$ can also be informed by the corresponding representation at layer l . The message calculated at layer $l+1$ can be represented as

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right),$$

where $h_i^{(l)}$ is the hidden representation of node i in the l -th layer of the neural network, $\mathcal{N}_i^r$ denotes the set of neighbor indices of node i with the edge type $r \in \mathcal{R}$, W_r is the weight for edge type r , W_0 is the weight for the self loop, and $c_{i,r}$ is a normalizing constant. Incoming messages are accumulated and passed through an element-wise activation function $\sigma(\cdot)$, we use $\text{ReLU}(\cdot) = \max(0, \cdot)$ as the activation function.

RGCN for Fraud Detection

There are multiple risk models and several checkpoints to detect fraudulent users at Uber. To potentially better serve these risk models, one idea is to derive fraud scores and use it as features in downstream risk models. The RGCN model outputs fraud scores for each user, indicating the user's risk. The fraud score learning flow is shown in Figure 3. The hidden representation of each node in the graph is learned to predict whether a user is fraudulent or



Engineering

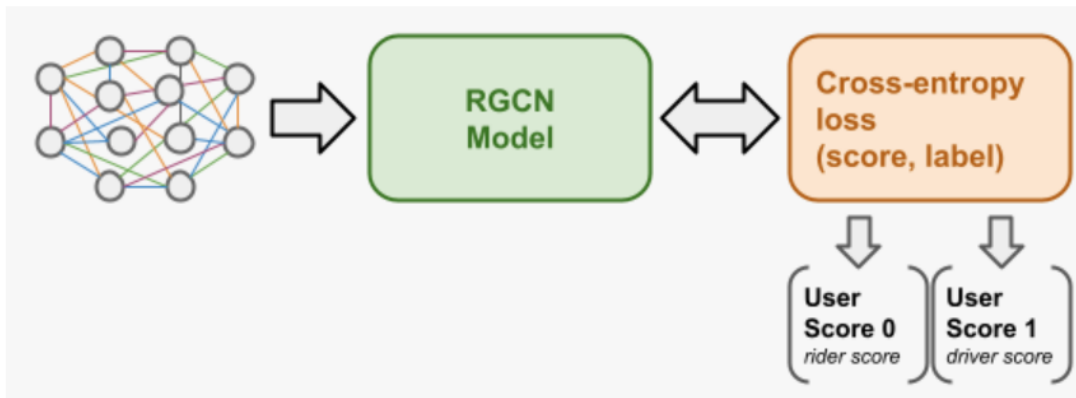


Figure 3: Fraud score modeling flow: score is learned via a binary cross-entropy loss. Model outputs are two scores, one for driver and one for rider.

We use two sources of inputs: node features (for users) and edge types. The driver-rider in-memory graph is constructed using the [DGL library](#) [7]. The label of fraud is whether a user has chargeback in a time range. We did feature engineering to help model learning. For example, a driver-rider graph has two types of nodes: drivers and riders. Each node type, driver or rider, may have different features. To solve that, we use zero padding to ensure the input feature vectors have the same size; second we define edge types specifically, and learn different weights for each type during model training.

To evaluate the RGCN model performance and utility of the fraud scores, we trained the model on historical data with 4 months length up to a specific split date. We then tested the model performance on the data from the 6 weeks following the split date. Specifically, we output fraud scores for users and computed the [Precision](#), [Recall](#), and [AUC](#). During the experiment we observed 15% better precision with minimal increase in false positives by adding two fraud score features to the existing production model. The two fraud scores are ranked 4th and 39th among the 200 features presented in the downstream model.

Engineering

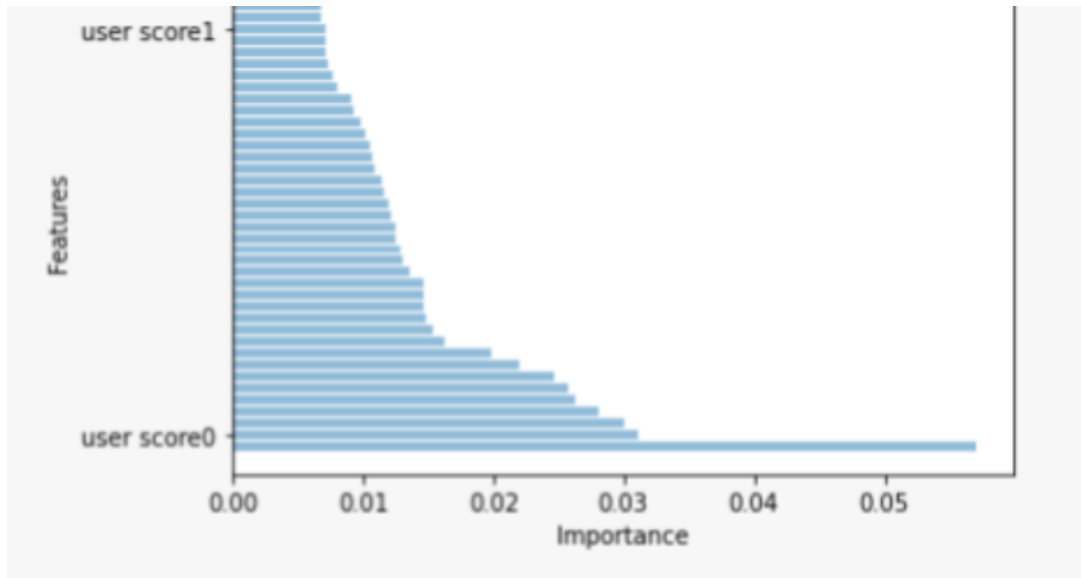


Figure 4: Feature importance of the features used in the downstream risk model: *user score1* and *user score0* learned from RGCN are ranked to be the 4th and the 39th respectively.

Data Pipeline

Data Ingestion

In a previous blog post on Food Discovery with Uber Eats [4], we explained how we leveraged our offline graph ecosystem to generate a city-level user-restaurant graph. In this use case, our requirement was to build one giant graph instead of several smaller city-level graphs. We reused many components like Cypher on Spark to generate a multi-relational user graph. Our ingestion framework transforms source hive tables into node and relationship tables. The node table captures user features whereas the relationship table captures different types of edges between users.

Graph Partitioning

The large size of the graph makes it necessary to use distributed training and prediction. The initial graph is partitioned into several smaller graphs such that it can fit into the memory of worker machines. We are only interested in the x-hop subgraph of users who have used the Uber platform recently. These recent “seed users” are then randomly assigned a partition number (0 to n). Each seed user’s x-hop subgraph is also pulled into the same partition. A user may be part of multiple partitions or a dormant user may not be in any partition. Each partition is mapped to a training/prediction worker machine.

We augmented the Cypher language to add a partition clause for graph creation. The below example query will automatically generate multiple graphs split by a partition column. Each partition will contain seed users and their one-hop neighborhood.



Engineering

```
CREATE (u)-[r1]->(u1)
RETURN GRAPH
```

Super Nodes

A big challenge to the graph generation process is handling super nodes, which is a node with an extremely high volume of connections. We manage this in two stages. First, while creating the relationship table, we filter entities with a high degree of connections. For example, two users connected by 1000 shared entities will lead to 10002 user-user relationships. However, we do add the counts as node features. Second, during the graph partitioning phase, there are some users with a very high degree of distinct relationships in their subgraph. This increases variance in partition size with some partitions being extremely large. We restrict such users to their first few hops depending on the threshold. These outlier cases can be tracked using rules.

Training & Batch Prediction

The data pipeline and training pipeline are shown in Figure 5. In the first step of the pipeline, data is pulled from Apache Hive tables and ingested into HDFS as Parquet files which contain node and edge information. Each node and edge is versioned by a timestamp. Graphs with the most recent properties of nodes and edges are retained given a specific date and stored in HDFS using Cypher format. Graphs are partitioned before feeding into the model by using the Cypher query language in an Apache Spark execution engine. Graph partitions are directly fed into DGL training and batch prediction applications. The generated scores are stored in Hive and used for actioning and offline analysis.

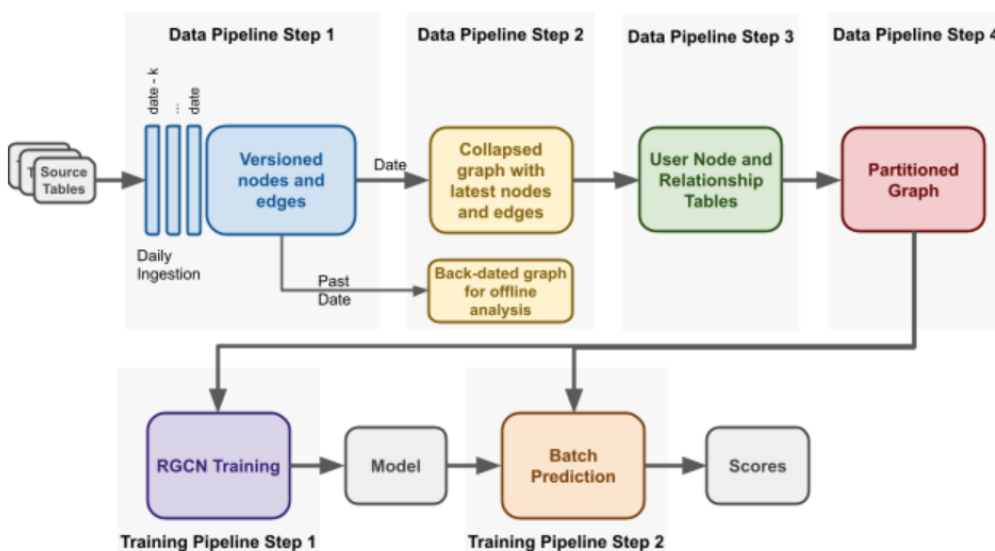


Figure 5: Data pipeline (top row) and training pipeline (bottom row) used to learn fraud scores for improving fraud detection.

Future Directions

Engineering

scalability and realtime-ness of our system. In particular, we are exploring a more efficient way to store massive graphs and conduct distributed training and real-time serving. In addition, as the driver-rider graphs are densely connected, to make message passing more efficient, we will explore attention-based graph models, which leverage masked self-attention layers and assign different importance to different nodes in a neighborhood. For example, graph attention networks [8] and a further extension of attending to far away neighbors [9] are relevant for our application.

Acknowledgements

The Uber AI, uGraph and Marketplace Risk team provided resources and support for implementation and experiments. Thanks for the consistent support and guidance from Qifa Ke, Long Sun and Santosh Golecha during the project development.

References

1. Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den, Ivan Titov, Max Welling, [Modeling Relational Data with Graph Convolutional Networks](#), ESWC 2018.
2. Ziqi Liu, Chaochao Chen et al., [Heterogeneous Graph Neural Networks for Malicious Account Detection](#), CIKM 2018.
3. Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton and Jure Leskovec, [Graph Convolutional Neural Networks for Web-Scale Recommender Systems](#), KDD 2018.
4. Ankit Jain, Isaac Liu, Ankur Sarda, and Piero Molino, [Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations](#).
5. Tahereh Pourhabibi, Kok-Leong Ong, Booi H.Kam, Yee Ling Boo, [Fraud detection: A systematic literature review of graph-based anomaly detection approaches](#), Decision Support Systems 2020.
6. Thomas N. Kipf and Max Welling, [Semi-Supervised Classification with Graph Convolutional Networks](#), ICLR 2017.
7. Minjie Wang, Lingfan Yu et al., [Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs](#), ICLR 2019.
8. Petar Velickovi, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, [Graph attention networks](#), ICLR 2018.
9. Kai Zhang, Yaokang Zhu, Jun Wang, Jie Zhang, [Adaptive structural fingerprints for graph attention networks](#), ICLR 2020.

Comments



Engineering

Xinyu Hu

Xinyu is a research scientist at the Uber AI engagement team. She works on projects on the intersection of machine learning, causal inference and graph learning. Xinyu holds a PhD in Biostatistics from Columbia University.

Chengliang Yang

Chengliang is a machine learning engineer in the Uber Risk engineering team. He primarily focuses on building machine learning based solutions to identify fraudulent behaviors on the Uber platform. Chengliang holds a PhD in Computer Science from the University of Florida with specialization in machine learning.

Lawrence Murray

Lawrence Murray is a research scientist manager at Uber AI. He works in probabilistic programming and Monte Carlo methods for Bayesian inference. Lawrence holds a PhD in Informatics from the University of Edinburgh.

Ankur Sarda

Ankur is a former software engineer at the Uber Risk engineering team focussing on graph applications at Uber.

Ankit Jain

Engineering

Piero Molino

Piero is a Staff Research Scientist in the Hazy research group at Stanford University. He is a former founding member of Uber AI where he created Ludwig, worked on applied projects (COTA, Graph Learning for Uber Eats, Uber’s Dialogue System) and published research on NLP, Dialogue, Visualization, Graph Learning, Reinforcement Learning and Computer Vision.

No posts to display

Get the App →

Engineering

Become a Driver

Contact Us

-  ubereng@uber.com
-  [@ubereng](#)
-  [UberEngineering](#)
-  [Uber Engineering](#)
-  [UberEngineering](#)
-  [UberEngineering](#)

Uber Engineering Blog Categories

- AI
- Architecture
- Culture
- General Engineering
- Mobile
- Open Source
- Uber Data

Uber Links

- Uber Open Source
- Uber Research
- Uber.com
- Uber Eats
- Uber for Business
- Help
- Newsroom
- Careers



Engineering

[Privacy Policy](#)

[Terms and Conditions](#)

